Zephyr Project

IT-Schulungen.com
Technology & Transformation

Anatomie einer Zephyr-Anwendung

Geführter Code-Walkthrough

1.09

Jede Datei hat eine klare Verantwortung

Fünf Perspektiven auf eine minimale Anwendung

01

Struktur

Anwendung außerhalb des Zephyr- Sourcetree organisieren

02

Build

CMake findet Zephyr und registriert Quellen

03

Konfiguration

prj.conf aktiviert benötigte Funktionen

04

Code

main() nutzt öffentliche Zephyr-APIs

05

Erweiterung

Overlay, Kconfig und Fragmente gezielt ergänzen

Ergebnis: Jede Kerndatei lässt sich ihrer Aufgabe im Build- und Laufzeitmodell zuordnen.

Minimale Out-of-tree-Struktur

PROJEKTROOT

my_app/

CMakeLists.txt

prj.conf

src/

Build-Ausgabe getrennt halten

SOURCES

src/main.c

Anwendungseinstieg

Zephyr-Header einbinden

öffentliche APIs verwenden

keine generierten Dateien

OPTIONAL

Bei Bedarf

app.overlay

Kconfig

boards/

zusätzliche .conf-Dateien

Out-of-tree bedeutet: Anwendung und eigener Build bleiben vom Zephyr-Sourcetree getrennt.

CMake verbindet App und Zephyr

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20.0)
find_package(Zephyr REQUIRED HINTS
$ENV{ZEPHYR_BASE})
project(my_app)
target_sources(app PRIVATE src/main.c)
```

01 find_package

Lädt Zephyrs Build-System und stellt das Anwendungs-Target bereit.

02 Target app

Alle eigenen Quellen werden diesem vorhandenen CMake-Target hinzugefügt.

Die Reihenfolge ist relevant: Erst Zephyr finden, dann das Projekt und seine Quellen beschreiben.

prj.conf formt die Konfiguration

KONFIGURATIONSFRAGMENT

```
CONFIG_LOG=y
CONFIG_PRINTK=y
# nur benötigte Optionen setzen
```

01 Fragment

Die Anwendung fordert benötigte Kconfig-Werte an.

02 Zusammenführen

Wertet Abhängigkeiten aus.

03 Ergebnis

.config enthält die tatsächlich wirksamen Werte.

prj.conf ist Eingabe – .config ist das geprüfte und zusammengeführte Ergebnis.

main() läuft als Zephyr-Thread

Einstiegspunkt	main() ist der Anwendungseinstieg und läuft im Zephyr-Main-Thread
Zephyr-Header	Öffentliche APIs werden über Header wie zephyr/kernel.h eingebunden
Warten	k_sleep() blockiert den Thread und gibt Rechenzeit für andere Threads frei
Rückkehr	Nach dem Rücksprung kann der Kernel weiterlaufen oder in Idle wechseln
Verantwortung	Anwendungslogik nutzt APIs; Kernel und Treiber bleiben Zephyr-Anteile

main() ist kein klassischer Hostprozess-Einstieg, sondern Teil des Zephyr-Threadmodells.

Optionale Dateien gezielt ergänzen

NUR WENN DIE ANWENDUNG ES BRAUCHT

Die Minimalstruktur bleibt klein; weitere Dateien lösen jeweils ein konkretes Problem.

app.overlay	ergänzt oder ändert die Hardwarebeschreibung für diese Anwendung
Kconfig	definiert eigene konfigurierbare Symbole und Abhängigkeiten
Zusätzliche .conf	bilden Varianten, Features oder Testkonfigurationen ab
boards/	enthält boardspezifische Overlays oder Konfigurationsfragmente
Build-Verzeichnis	bleibt generierte Ausgabe und gehört nicht zu den Quellen

Jede optionale Datei braucht eine begründete Verantwortung – nicht bloß einen bekannten Namen.

Sample und eigene Anwendung vergleichen

ZWEI ORTE – GLEICHES MODELL

```
Zephyr-Repository:
  samples/hello_world/

Eigener Workspace:
  applications/my_app/
```

Kern: CMakeLists.txt · prj.conf · src/main.c

01 Sample im Zephyr-Tree

- Lehr- und Referenzcode
- Version und Tests sind an das Zephyr-Repository gekoppelt.

02 Out-of-tree-App

- Eigenständig versioniert
- Zephyr wird als Abhängigkeit des Projekts verwendet.

Der Ort ändert sich – die Rollen von Build-Datei, Konfiguration und Quelltext bleiben gleich.

Anatomie verstanden

KERNDATEN

Verantwortung geklärt

CMakeLists.txt verbindet den Build

prj.conf konfiguriert

src/main.c implementiert

Build bleibt generiert

ZUORDNUNG

Datei oder Ergebnis?

Quelltext benennen

Konfiguration erklären

Hardwarebeschreibung erkennen

Build-Artefakte abgrenzen

NÄCHSTER SCHRITT

Praxis 1.10

Projektverzeichnis anlegen

Kerndateien erstellen

Statusausgabe ergänzen

für native_sim bauen

Lernziel: Zweck und Verantwortlichkeit der zentralen Anwendungsdateien erklären.